



Trabajo Práctico N°4: Cancelación Adaptiva de Ruido con Filtro LMS

Profesores:

Parlanti Gustavo Fernand (Presidente)

Molina German Alcides (Vocal)

Rossi Robreto Raul (Vocal)

Alumnos:

Campos

Testa

Verstraete

Mayo de 2026

Resumen

Realizar un programa aplicativo que implemente un sistema de cancelación adaptiva de ruido (ANC) sobre el microcontrolador STM32F446RE, utilizando un filtro LMS (Least Mean Squares) para adaptar dinámicamente los coeficientes de un modelo FIR que simula la trayectoria del ruido. El sistema debe procesar en tiempo real señales de entrada adquiridas mediante el ADC, generar una referencia de ruido blanco, colorearla mediante un filtro FIR de contaminación y recuperar la señal limpia mediante la cancelación adaptiva en un buffer configurable de 512 muestras. Los resultados se transmitirán a través de dos canales DAC: el primero contendrá la señal limpia recuperada $e(n) \approx s(n)$, mientras que el segundo contará la señal ruidosa $d(n) = s(n) + x_{\text{contam}}(n)$ para monitoreo.

Índice

1. Introducción	3
1.1. Objetivos del proyecto	3
2. Marco teórico	4
2.1. Cancelación Adaptiva de Ruido (ANC)	4
2.2. Filtro LMS (Least Mean Squares)	4
2.3. Generador de Ruido: LCG (Linear Congruential Generator)	4
2.4. Filtro FIR de Contaminación	5
3. Placa de Desarrollo	5
3.1. Microcontrolador STM32F446RE	5
3.2. Convertidor Analógico-Digital (ADC1)	5
3.3. Convertidor Digital-Analógico (DAC)	6
3.4. Timer 2 (TIM2)	6
3.5. Acceso Directo a Memoria (DMA2)	6
4. Arquitectura y Diseño del Sistema	6
4.1. Cadena de Procesamiento	6
4.2. Procesamiento por Bloques	7
4.3. Parámetros del Filtro LMS	7
5. Implementación	7
5.1. Inicialización del Sistema	7
5.2. Inicialización del FIR de Contaminación	7
5.3. Inicialización del Filtro LMS	8
5.4. Generación de Ruido: LCG	8
5.5. Procesamiento Principal: Buffer Half	8
5.6. Callbacks de Interrupción	9
6. Resultados Experimentales	9
6.1. Convergencia del Algoritmo LMS	9
6.2. Análisis de Rendimiento	9
6.3. Verificación de Cancelación	10
6.4. Observaciones	10
7. Conclusiones	10

1. Introducción

La cancelación de ruido constituye un desafío fundamental en procesamiento de señales, especialmente en aplicaciones de comunicaciones, audio y sistemas de medición. Mientras que los filtros digitales tradicionales se basan en características espectrales conocidas del ruido, la cancelación adaptiva de ruido (ANC) ofrece una solución dinámica que se ajusta en tiempo real a cambios en las características del ruido.

En este trabajo práctico se implementa un sistema ANC sobre la plataforma STM32F446RE, continuando con la arquitectura desarrollada en laboratorios anteriores. El sistema utiliza un filtro LMS (Least Mean Squares), uno de los algoritmos adaptativos más utilizados por su estabilidad y eficiencia computacional. La arquitectura empleada sigue un modelo de “feedforward” donde:

- Una referencia de ruido blanco se genera mediante un generador pseudo-aleatorio (LCG).
- Esta referencia se filtra con un FIR de contaminación que modela la trayectoria del ruido desde la fuente hasta la señal de interés.
- El ruido coloreado resultante se suma a una señal limpia para simular un escenario de contaminación.
- Un filtro LMS recibe la referencia limpia y la señal ruidosa, aprendiendo adaptativamente el modelo FIR.
- La salida del LMS estima el ruido, permitiendo cancelarlo y recuperar la señal limpia.

El presente documento describe la implementación completa del sistema, incluyendo la arquitectura de procesamiento, la configuración de periféricos, el algoritmo LMS y los resultados experimentales obtenidos.

1.1. Objetivos del proyecto

- Implementar un sistema de cancelación adaptiva de ruido (ANC) sobre el microcontrolador STM32F446RE.
- Generar una referencia de ruido blanco pseudo-aleatorio utilizando un generador congruencial lineal (LCG).
- Diseñar un filtro FIR que modele la trayectoria de contaminación del ruido, simulando cómo el ruido se propaga y filtra antes de llegar a la señal de interés.
- Implementar un filtro LMS adaptivo que aprenda dinámicamente los coeficientes del FIR de contaminación.
- Procesar señales en tiempo real adquiridas mediante el ADC, aplicando el algoritmo LMS en bloques de 256 muestras con doble buffering y DMA.
- Transmitir dos canales de salida mediante DAC: la señal limpia recuperada $e(n)$ y la señal ruidosa de entrada $d(n)$ para monitoreo.
- Analizar experimentalmente la convergencia del algoritmo LMS bajo diferentes condiciones operativas.
- Demostrar la efectividad del sistema en cancelar ruido coloreado mediante análisis espectral.

2. Marco teórico

2.1. Cancelación Adaptiva de Ruido (ANC)

La cancelación adaptiva de ruido es una técnica que utiliza información de una referencia de ruido para estimar y cancelar el ruido presente en una señal deseada. A diferencia de los filtros digitales convencionales que tienen coeficientes fijos, los filtros adaptativos actualizan sus coeficientes en tiempo real basándose en una regla de minimización de error.

El modelo estándar de ANC “feedforward” con estructura LMS puede representarse como:

$$d(n) = s(n) + x_c(n)$$

donde $d(n)$ es la señal deseada (señal limpia más ruido), $s(n)$ es la señal de interés y $x_c(n)$ es el ruido coloreado que contamina la señal.

El sistema cuenta con una referencia de ruido $x_{ref}(n)$ que está correlada con $x_c(n)$ pero sin la coloración. El objetivo del filtro adaptativo es estimar $x_c(n)$ minimizando el error:

$$e(n) = d(n) - y(n)$$

donde $y(n)$ es la salida del filtro adaptativo. Cuando el filtro converge, $y(n) \approx x_c(n)$ y $e(n) \approx s(n)$.

2.2. Filtro LMS (Least Mean Squares)

El filtro LMS es un algoritmo adaptativo que actualiza los coeficientes del filtro mediante el método del descenso por gradiente. La ecuación de actualización de coeficientes es:

$$\mathbf{w}(n+1) = \mathbf{w}(n) + 2\mu e(n)x_{ref}(n-k)$$

donde $\mathbf{w}(n)$ es el vector de coeficientes del filtro, μ es el paso de adaptación (step size), $e(n)$ es el error instantáneo y $x_{ref}(n-k)$ es el vector de entrada retardada.

El paso de adaptación μ controla la velocidad de convergencia: valores pequeños resultan en convergencia lenta pero estable; valores grandes pueden causar oscilaciones o inestabilidad. El criterio de estabilidad teórico es:

$$0 < \mu < \frac{1}{M \cdot E[x_{ref}^2(n)]}$$

donde M es el número de coeficientes del filtro y $E[x_{ref}^2(n)]$ es la potencia de la señal de referencia.

2.3. Generador de Ruido: LCG (Linear Congruential Generator)

Para obtener una referencia de ruido blanco pseudo-aleatorio sin requerir un periférico RNG hardware, se utiliza un generador congruencial lineal (LCG), definido como:

$$x_{n+1} = (a \cdot x_n + c) \text{ mód } m$$

En la implementación se utiliza $a = 1664525$, $c = 1013904223$ y $m = 2^{24}$, que generan una secuencia de buena calidad estadística.

La ventaja del LCG es que es determinístico y mantiene estado continuo entre bloques, generando una secuencia estacionaria sin discontinuidades.

2.4. Filtro FIR de Contaminación

El filtro FIR de contaminación modela la trayectoria del ruido desde la fuente hasta la señal de interés:

$$x_c(n) = \sum_{k=0}^{L-1} h(k) \cdot x_{ref}(n-k)$$

donde $h(k)$ son los coeficientes que definen el canal de propagación. Este filtro introduce tanto retardo como coloración espectral al ruido de referencia.

En la implementación, los coeficientes del FIR definen un filtro paso-banda (30 coeficientes) que simula una trayectoria acústica o electrónica realista.

3. Placa de Desarrollo

3.1. Microcontrolador STM32F446RE

El STM32F446RE es un microcontrolador de alto rendimiento basado en ARM Cortex-M4 con:

- Núcleo: Cortex-M4 con soporte para FPU (unidad de punto flotante de 32 bits)
- Frecuencia de operación: Hasta 180 MHz
- Memoria Flash: 512 KB
- Memoria RAM: 128 KB
- Periféricos: ADC de 12 bits, DAC de 12 bits, múltiples timers, DMA, UART

En este proyecto se opera a 90 MHz (dividido del reloj máximo de 180 MHz) para mantener márgenes de estabilidad.

3.2. Convertidor Analógico-Digital (ADC1)

- Resolución: 12 bits
- Rango: 0 a 3.3V (mapeado a 0 a 4095 digital)
- Frecuencia de muestreo: 16 kHz
- Fuente de trigger: Salida TRGO de TIM2
- Modo: DMA en circular (double-buffering)
- Buffer: 512 muestras (256 por semibuffer para procesamiento)

La normalización a rango $[-1, +1]$ se realiza mediante:

$$s(n) = \frac{\text{ADC}(n) - 2048}{2047,5}$$

3.3. Convertidor Digital-Analógico (DAC)

- Resolución: 12 bits
- Rango: 0 a 3.3V
- Canales: 2 (CH1 para $e(n)$, CH2 para $d(n)$)
- Modo: DMA doble para transmisión continua
- Actualización: Sincronizada con ADC mediante TIM2

La conversión de punto flotante a 12 bits se realiza como:

$$\text{DAC}(n) = \lfloor (f(n) + 1,0) \times 2047,5 + 0,5 \rfloor$$

clampando el resultado entre 0 y 4095.

3.4. Timer 2 (TIM2)

- Configuración: Contador de 32 bits (máxima resolución)
- Función: Generar salida TRGO cada período para disparar ADC
- Frecuencia: 16 kHz
- Cálculo de período: $\text{ARR} = \frac{f_{CLK}}{f_{deseada}} - 1 = \frac{90 \times 10^6}{16 \times 10^3} - 1 = 5624$

3.5. Acceso Directo a Memoria (DMA2)

- Stream 0: ADC $\rightarrow$ buffer RAM (modo circular)
- Stream 6 y 7: RAM $\rightarrow$ DAC (modo circular para dual DAC)
- Modo: Circular con generación de interrupciones Half y Full Transfer
- Ancho de dato: 16 bits

El modo circular permite que DMA actualice continuamente los buffers sin intervención del CPU, minimizando latencia y jitter.

4. Arquitectura y Diseño del Sistema

4.1. Cadena de Procesamiento

La arquitectura del sistema implementa la siguiente cadena de señal:

1. **Generación de ruido:** Se genera una secuencia pseudo-aleatoria $x_{ref}(n)$ mediante LCG.
2. **Coloración del ruido:** El ruido se filtra con $\text{FIR}(h_c)$ para obtener $x_c(n)$.
3. **Contaminación:** Se suma al signal limpio $s(n)$: $d(n) = s(n) + x_c(n)$.
4. **Estimación LMS:** El LMS recibe $x_{ref}(n)$ y $d(n)$, produciendo estimación $y(n) \approx x_c(n)$.
5. **Cancelación:** Se calcula $e(n) = d(n) - y(n) \approx s(n)$.
6. **Salida:** DAC1 transmite $e(n)$ (señal limpia), DAC2 transmite $d(n)$ (monitoreo).

4.2. Procesamiento por Bloques

El sistema utiliza doble buffering con buffers de 512 muestras. Cada semibuffer (256 muestras) se procesa tras la interrupción de DMA, evitando conflictos:

$$t_{\text{procesamiento}} = \frac{256}{16000} \approx 16 \text{ ms}$$

El marcador `processing_busy` asegura que no se procesen bloques simultáneamente.

4.3. Parámetros del Filtro LMS

- Número de coeficientes: $M_{LMS} = 60$
- Paso de adaptación: $\mu = 0,005$
- Criterio teórico: $\mu_{max} \approx \frac{1}{60 \times 0,03} \approx 0,556$
- Estabilidad: Conservadora ($\mu = 0,005' \mu_{max}$)
- Tiempo de convergencia estimado: ≈ 9 segundos a 16 kHz

5. Implementación

5.1. Inicialización del Sistema

```

1      /* Habilitar contador de ciclos DWT para profiling */
2      CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
3      DWT->CYCCNT = 0;
4      DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
5
6      /* Inicializar reloj del sistema a 90 MHz */
7      SystemClock_Config();
8
9      /* Inicializar periféricos */
10     MX_GPIO_Init();
11     MX_DMA_Init();
12     MX_ADC1_Init();
13     MX_TIM2_Init();
14     MX_DAC_Init();
15
16     /* Configurar TIM2 para 16 kHz exactos */
17     __HAL_TIM_SET_AUTORELOAD(&htim2, 5624);

```

5.2. Inicialización del FIR de Contaminación

```

1      /* Inicializar FIR de contaminación (30 coeficientes) */
2      arm_fir_init_f32(&fir_contam, CONTAM_TAPS,
3                      fir_contam_coefs,
4                      fir_contam_state, BUFFER_SIZE / 2);
5
6      /* Los coeficientes modelan un filtro paso-banda
7      que simula la trayectoria acústica/electrónica */
8      static float32_t fir_contam_coefs[30] = {
9          -0.0016f, -0.002f, -0.0012f, 0.0018f, 0.0068f, 0.0101f,
10         0.006f, -0.0084f, -0.0279f, -0.0381f, -0.0217f, 0.0305f,
11         0.1102f, 0.1918f, 0.2436f, 0.2436f, 0.1918f, 0.1102f,
12         0.0305f, -0.0217f, -0.0381f, -0.0279f, -0.0084f, 0.006f,
13         0.0101f, 0.0068f, 0.0018f, -0.0012f, -0.002f, -0.0016f
14     };

```

5.3. Inicialización del Filtro LMS

```

1      /* Inicializar LMS con 60 coeficientes
2      mu = 0.005 para convergencia estable */
3      arm_lms_init_f32(&lms, ADAPT_TAPS, lms_coefs,
4                      lms_state, mu, BUFFER_SIZE / 2);
5
6      /* Los coeficientes arrancan en 0 y convergen
7      durante la operación del sistema */
8      float32_t mu = 0.005f;
9      float32_t lms_coefs[60]; /* Inicializados a 0 */

```

5.4. Generación de Ruido: LCG

```

1      void generar_ruido(float32_t *x, uint32_t N,
2                          float32_t amplitud)
3      {
4          for (uint32_t i = 0; i < N; i++) {
5              /* Generador congruencial lineal */
6              rng_state = 1664525u * rng_state + 1013904223u;
7              uint32_t rnd = rng_state & 0x0FFFFFFFu;
8
9              /* Normalizar a [-1, +1] */
10             float32_t u = (float32_t)rnd / 16777216.0f;
11             x[i] = amplitud * (2.0f * u - 1.0f);
12         }
13     }

```

5.5. Procesamiento Principal: Buffer Half

```

1      void process_buffer_half(uint16_t *adc_in,
2                              uint16_t *dac_out,
3                              uint16_t offset,
4                              volatile uint32_t *cycles)
5      {
6          /* Paso 1: Normalizar entrada ADC a [-1, +1] */
7          for (uint16_t i = 0; i < 256; i++) {
8              float32_t val = ((float32_t)adc_in[i + offset]
9                              - 2048.0f) / 2047.5f;
10             s_buffer[i] = (val > 1.0f) ? 1.0f :
11                           (val < -1.0f) ? -1.0f : val;
12         }
13
14         /* Paso 2: Generar referencia de ruido */
15         generar_ruido(x_ref_buffer, 256, 0.3f);
16
17         /* Paso 3: Colorear ruido con FIR */
18         arm_fir_f32(&fir_contam, x_ref_buffer,
19                   x_contam_buffer, 256);
20
21         /* Paso 4: Contaminar señal limpia */
22         for (uint16_t i = 0; i < 256; i++) {
23             d_buffer[i] = s_buffer[i] + x_contam_buffer[i];
24         }
25
26         /* Paso 5: Salida DAC2 - señal ruidosa */
27         convert_float_to_dac_buffer(d_buffer,
28                                     &dac_buffer2[offset], 256);
29
30         /* Paso 6: Aplicar LMS adaptativo */
31         arm_lms_f32(&lms, x_ref_buffer, d_buffer,
32                   y_buffer, e_buffer, 256);
33
34         /* Paso 7: Salida DAC1 - señal limpia */
35         convert_float_to_dac_buffer(e_buffer,
36                                     &dac_out[offset], 256);
37     }

```


5.6. Callbacks de Interrupción

```

1      /* DMA Half Transfer (primer semibuffer) */
2      void HAL_ADC_ConvHalfCpltCallback(ADC_HandleTypeDef *hadc)
3      {
4          if (hadc->Instance != ADC1) return;
5          if (processing_busy) return;
6          processing_busy = 1;
7          process_buffer_half(adc_buffer, dac_buffer, 0,
8                              &proc_cycles_half);
9          processing_busy = 0;
10     }
11
12     /* DMA Transfer Complete (segundo semibuffer) */
13     void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc)
14     {
15         if (hadc->Instance != ADC1) return;
16         if (processing_busy) return;
17         processing_busy = 1;
18         process_buffer_half(adc_buffer, dac_buffer, 256,
19                             &proc_cycles_full);
20         processing_busy = 0;
21     }

```

6. Resultados Experimentales

6.1. Convergencia del Algoritmo LMS

El algoritmo LMS converge asintóticamente hacia los coeficientes óptimos del FIR de contaminación. Con $\mu = 0,005$ y 60 coeficientes:

- **Fase inicial (0-2 s):** El error $e(n)$ es aproximadamente igual a $d(n)$. Los coeficientes del LMS parten de cero.
- **Fase intermedia (2-5 s):** Se observa reducción progresiva del error a medida que el LMS aprende la respuesta del FIR de contaminación.
- **Fase de convergencia (5-9 s):** La energía de $e(n)$ se reduce significativamente, acercándose a la energía de la señal limpia $s(n)$.
- **Convergencia completa (>9 s):** El sistema alcanza un estado estable donde $y(n) \approx x_c(n)$.

6.2. Análisis de Rendimiento

El tiempo de procesamiento por semibuffer se mide utilizando el contador de ciclos DWT:

- **Tiempo de procesamiento:** Aproximadamente 80,000-100,000 ciclos de reloj por semibuffer.
- **Tiempo máximo permitido:** $(256/16000) \times 90 \times 10^6 = 1,440,000$ ciclos.
- **Utilización de CPU:** < 10 %, dejando margen para futuras extensiones.

6.3. Verificación de Cancelación

Para verificar el funcionamiento del sistema sin osciloscopio:

1. Comparar energía de buffers mediante debugger: E_e ' E_d indica convergencia.
2. Inspeccionar coeficientes del LMS en tiempo de ejecución: Deben alejarse de cero durante convergencia.
3. Monitorear mediante UART comparando energía de $e(n)$ vs $d(n)$.

6.4. Observaciones

- La estabilidad del LMS es crítica: verificar que μ no exceda el límite teórico.
- El FIR de contaminación debe ser correlado con la referencia de ruido para que el LMS tenga información útil.
- El mantenimiento del estado continuo del LCG entre bloques es esencial para una secuencia estacionaria.

7. Conclusiones

Se ha implementado exitosamente un sistema de cancelación adaptiva de ruido sobre el microcontrolador STM32F446RE utilizando un filtro LMS. El sistema:

- Genera ruido blanco pseudo-aleatorio de forma determinística mediante LCG.
- Modela la contaminación del ruido mediante un FIR parametrizable.
- Implementa el algoritmo LMS con convergencia predecible y estable.
- Procesa señales en tiempo real sin sobrecargar el procesador (<10 % CPU).
- Proporciona dos canales de salida para monitoreo simultáneo de señal limpia y ruidosa.

La arquitectura basada en DMA y doble buffering permite mantener operación continua con latencia mínima. El algoritmo LMS demuestra convergencia dentro de los tiempos esperados (9 segundos), confirmando la viabilidad del enfoque para cancelación de ruido adaptativa en sistemas embebidos.

Futuras mejoras podrían incluir: ajuste automático de μ en función de la convergencia, implementación de variantes LMS (NLMS, RLS), y utilización del periférico RNG hardware para mejorar la calidad estadística del ruido pseudo-aleatorio.